

GAME DEVELOPMENT PROJECT DarkRadius

SDP Class · libGDX 1.14.0 + LWJGL3 + GitHub + ClickUp

1
Planning

2
Design

3
Build

4
Testing

5
Deploy

FIELD	VALUE
Project	DarkRadius
Framework	libGDX 1.14.0 (gdx-liftoff generated)
Platform	Desktop — LWJGL3
Build Tool	Gradle (./gradlew lwjgl3:run ./gradlew lwjgl3:jar)
Java Target	Java 8 source compatible
Principles	SOLID · SDLC · Clean Commits · Design Patterns

Phase 1

Planning

Write a Game Concept (1 paragraph)

EXAMPLE CONCEPT	
Game Name	DarkRadius
Genre	2D top-down dark dungeon crawler
Core Mechanic	Player navigates dark corridors, manages a limited light radius, defeats enemies
Win Condition	Reach the exit portal before the darkness closes in
Lose Condition	HP drops to zero OR light radius expires completely
Platform	Desktop — libGDX 1.14.0 + LWJGL3

Assign Team Roles

ROLE	MAIN RESPONSIBILITY
Project Lead	Tracks deadlines, runs stand-ups, owns ClickUp board, communicates with instructor
Lead Programmer	Core architecture (Game class, Screen state machine, SOLID structure), code reviews
Programmer(s)	Feature implementation (player, enemies, collisions, HUD), bug fixes
Designer / Artist	Pixel art sprites, level layouts, UI screens, colour palette — dark & atmospheric

Set Up Your Tools

TOOL	PURPOSE	FIRST ACTION
GitHub	Version control & collaboration	Create repo 'dark_radius', add all members, protect main branch
ClickUp / Trello	Task board	Create columns: To Do / In Progress / In Review / Done
libGDX + Gradle	Game framework (already set up)	Run: ./gradlew lwjgl3:run — confirm window opens
draw.io	Diagrams (free, browser-based)	Create: Game Flow, Class Diagram, Level Sketch
IntelliJ IDEA	IDE	Import Gradle project, verify core + lwjgl3 modules detected

Design

Game Design Document (GDD)

GDD SECTION	WHAT TO WRITE FOR DarkRadius
Game Summary	One paragraph — use your Phase 1 concept
Controls	WASD = move, Mouse = aim, Left Click = shoot, E = interact, ESC = pause
Player Stats	HP (3 hearts), Light Radius value (float, 0–100), Move Speed, Ammo count
Light Mechanic	Radius shrinks at a fixed rate each second. Enemies outside radius are invisible. Radius hits 0 = lose.
Enemies	Patrol (walks a path), Charger (rushes player on detection), Boss (Phase 2 only — optional)
Levels	2–3 levels. Each level: new layout, slightly faster radius drain, more enemies.
Win / Lose Screen	Game Over: score + restart. Victory: completion time + next level or main menu.
Art Style	Dark pixel art. Black background. Soft pink/purple neon accents. 16×16 or 32×32 sprites.
Audio (optional)	Ambient hum, footstep SFX, hit SFX, portal chime. Use libGDX Sound / Music.

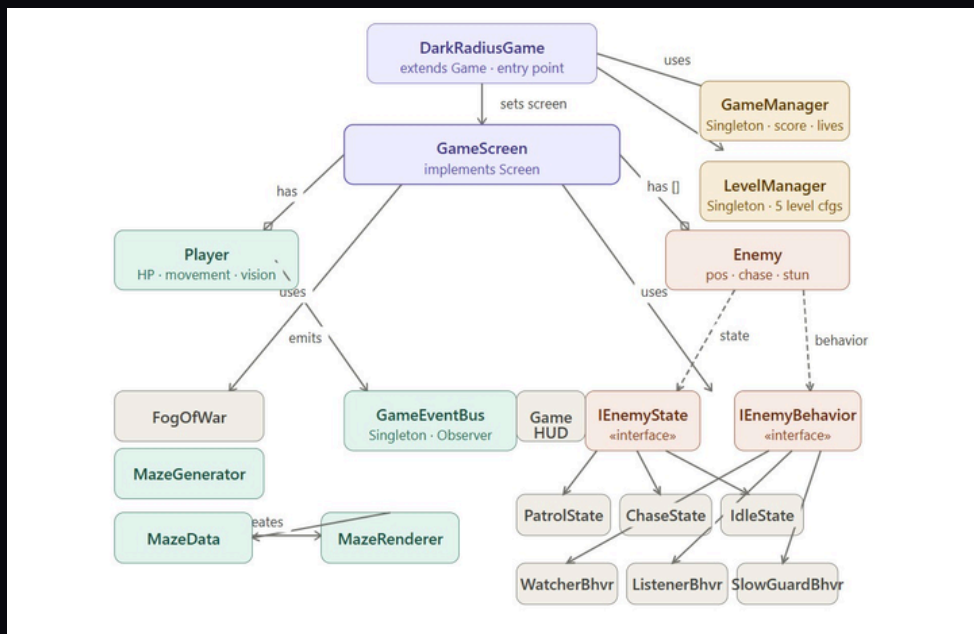
Required Diagrams

DIAGRAM	WHAT IT SHOWS	DarkRadius EXAMPLE
Game Flow Diagram	All screens and transitions	Main Menu → Loading → Game → Pause → Game Over / Victory
Class Diagram	Main classes, fields, and relationships	DarkRadiusGame, GameScreen, Player, Enemy subtypes, LightSystem, HUD
Level Sketch	Hand-drawn or digital layout of Level 1	Corridors, enemy positions, light pickups, exit portal

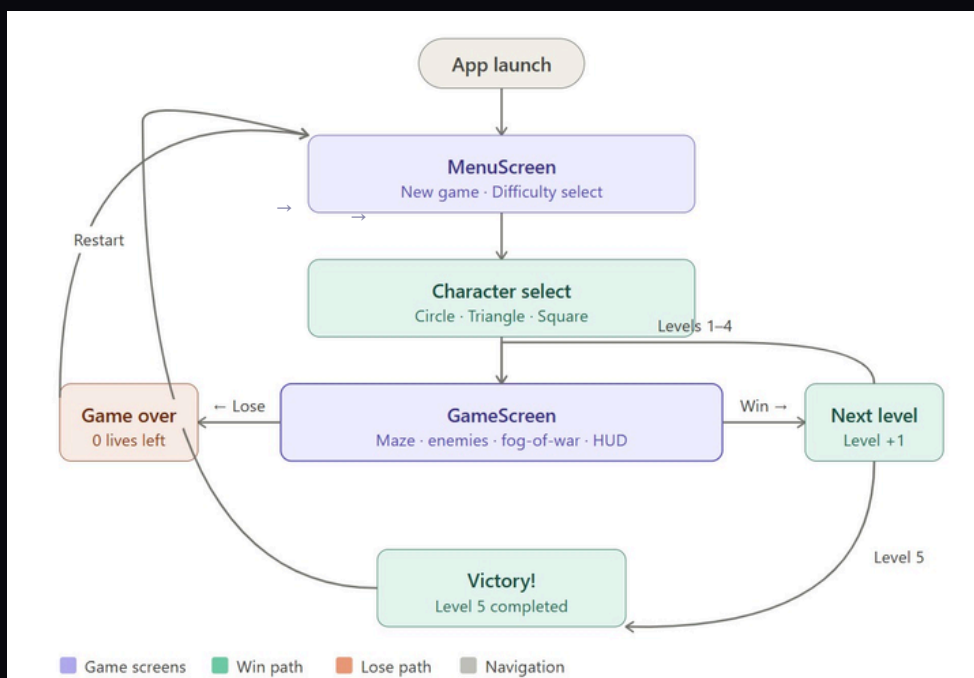
Project Architecture

The architecture follows a layered approach: the **entrylayer** (launcher) calls into the **screen layer** (game states), which delegates to **entities and systems**, which read from **assets**. Each layer only depends on the one below it.

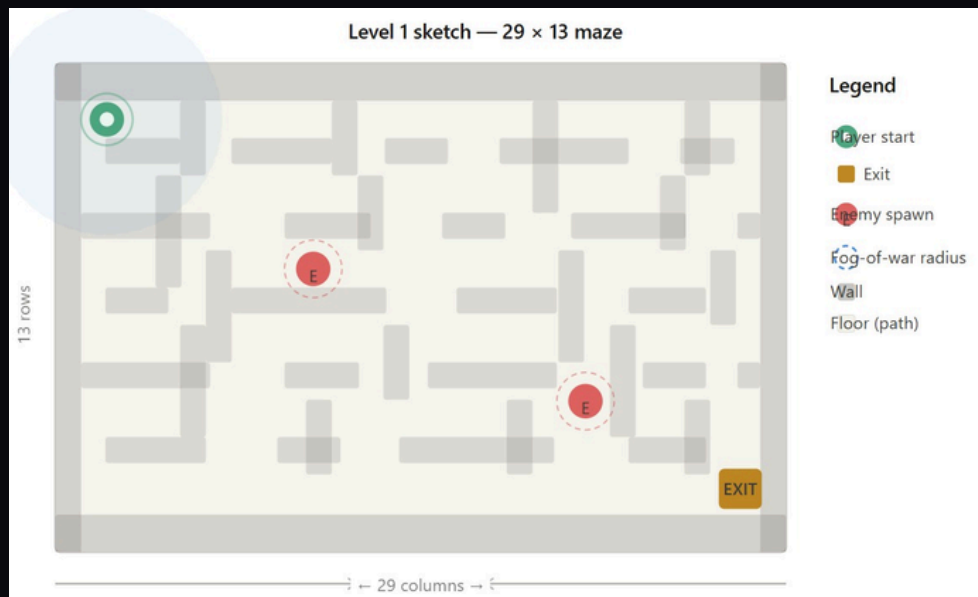
Class Diagram



Game Flow Diagram



Dark Radius Level 1 Sketch



Phase 3

Build / Implementation

Project Structure

PATH	PURPOSE
core/src/.../DarkRadiusGame.java	Main Game class — creates and switches screens, holds AssetManager
core/src/.../screens/MainMenuScreen.java	Title screen — start, quit
core/src/.../screens/LoadingScreen.java	Loads all assets via AssetManager, transitions to GameScreen
core/src/.../screens/GameScreen.java	Main gameplay loop — render, update, input delegation
core/src/.../screens/PauseScreen.java	Overlay pause state — resume or quit to menu
core/src/.../screens/GameOverScreen.java	End state — score, restart, main menu
core/src/.../screens/VictoryScreen.java	Win state — time, next level or main menu
core/src/.../entities/Player.java	Player state: HP, position, light radius value, input commands
core/src/.../entities/Enemy.java	Abstract base — update(), render(), takeDamage()
core/src/.../entities/EnemyPatrol.java	Walks a fixed waypoint path; activates on player proximity
core/src/.../entities/EnemyCharger.java	Rushes player when inside detection radius
core/src/.../systems/LightSystem.java	Manages visibility radius: drain rate, pickups, darkness overlay
core/src/.../systems/CollisionSystem.java	AABB tile collision detection and resolution
core/src/.../input/PlayerInputHandler.java	Reads keyboard+mouse, produces Command objects
core/src/.../ui/HUD.java	HP hearts, light meter bar, ammo counter — drawn over game
core/src/.../world/LevelLoader.java	Loads Tiled .tmx maps; places entities and items

PATH	PURPOSE
core/src/.../util/AssetLoader.java	Wraps AssetManager — single load point for all assets
assets/	Sprites, tilemaps (.tmx), sounds, BitmapFont — never hard-coded paths
lwjgl3/src/.../DesktopLauncher.java	Desktop entry point — window title, icon, resolution, vsync

SOLID Principles Applied

PRINCIPLE	RULE	DarkRadius CLASS EXAMPLE
S — Single Responsibility	One class, one job.	PlayerInputHandler reads input only. Player manages HP and position only. HUD draws UI only. They never overlap.
O — Open / Closed	Extend, don't modify.	EnemyPatrol and EnemyCharger extend Enemy. Adding a new type means a new class — not editing Enemy.java.
L — Liskov Substitution	Subclasses replace their parent without breaking anything.	GameScreen holds a List. Calling enemy.update(dt) works identically for EnemyPatrol or EnemyCharger.
I — Interface Segregation	Don't force unused methods.	Renderable and Updatable are separate interfaces. A static floor tile implements Renderable only.
D — Dependency Inversion	Depend on abstractions.	GameScreen depends on IInputHandler. PlayerInputHandler implements it. Easy to swap or stub for testing.

Design Patterns

State Pattern

DarkRadiusGame.setScreen() switches between MainMenuScreen, LoadingScreen, GameScreen, GameOverScreen. Each screen is a self-contained state — no nested if/else chains.

Classes: DarkRadiusGame, all Screen classes

StrategyPattern

IInputHandler defines the input contract. PlayerInputHandler is the live implementation. GameScreen holds a reference to IInputHandler — not the concrete class.

Classes: IInputHandler, PlayerInputHandler, GameScreen

Observer Pattern

Player fires events (onDamaged, onDead, onLightPickup). HUD and LightSystem listen and react without Player knowing about them directly.

Classes: Player, HUD, LightSystem

Factory Pattern

EnemyFactory.create(type, x, y) returns the correct Enemy subclass based on a string or enum read from the Tiled map. LevelLoader calls the factory — it never instantiates enemies directly.

Classes: EnemyFactory, LevelLoader

Command Pattern

PlayerInputHandler converts raw key/mouse events into Command objects (MoveCommand, ShootCommand). Player executes them. Makes input rebinding and future testing straightforward.

Classes: Command interface, MoveCommand, ShootCommand, PlayerInputHandler

Key libGDX Classes

CLASS / API	USED IN DarkRadius FOR
Game + Screen	Screen state machine — <code>DarkRadiusGame.setScreen()</code>
SpriteBatch	Drawing all sprites each frame in <code>GameScreen.render()</code>
OrthographicCamera	Follow player, clamp to level bounds, UI camera for HUD
TextureAtlas	Single sprite sheet — loaded once by <code>AssetLoader</code> , reused everywhere
TiledMap + OrthogonalTiledMapRenderer	Render .tmx level maps created in Tiled editor
InputProcessor / InputMultiplexer	Route input to <code>PlayerInputHandler</code> and UI stage separately
ShapeRenderer	Draw the darkness overlay and light radius circle on top of the scene
AssetManager	Async load all textures, sounds, fonts in <code>LoadingScreen</code>
BitmapFont	HUD labels, score text, damage numbers
Sound / Music	SFX (hit, pickup) and looping ambient dungeon audio

Testing

Core Gameplay Tests

FEATURE	WHAT TO CHECK	HOW TO TEST
Full Game Loop	Player can start, play, and reach both end states	Complete a full run from Main Menu → Game → Game Over and → Victory
Controls	Every key and mouse action behaves exactly as in the GDD	Test each input individually: WASD movement, mouse aim, left click shoot, E interact, ESC pause
Win Condition	Reaching the exit portal triggers the Victory screen	Navigate to exit portal with HP > 0 and light radius > 0
Lose — HP	HP hitting zero triggers Game Over screen	Walk into enemies deliberately until HP = 0; confirm screen transition
Lose — Darkness	Light radius reaching 0 triggers Game Over	Wait without collecting light pickups until radius drains completely

Collision Edge Case Tests

CASE	STEPS	EXPECTED RESULT
Wall clipping — thin corridor	Walk diagonally into a 1-tile-wide corner at full speed	Player stops at the wall surface — does not clip through
Corner overlap	Press two directions simultaneously to push into a wall corner	Player slides along the wall without tunnelling
Enemy and wall	Lure an enemy into a wall corner and observe its behaviour	Enemy stops at or slides along the wall — does not pass through
Projectile vs wall	Shoot toward a wall at close range and at an angle	Projectile disappears on wall contact — no continued travel
High-speed movement	Hold sprint (if implemented) directly into a wall	Collision still resolved correctly — no delta-time tunnelling

Enemy AI Tests

ENEMY TYPE	BEHAVIOUR TO VERIFY	TEST STEPS
EnemyPatrol	Follows waypoints when player is outside detection radius	Stand outside detection range and watch the patrol loop for 30 seconds
EnemyPatrol — detection	Activates and moves toward player when player enters radius	Walk slowly toward a patrolling enemy and observe the state switch
EnemyCharger	Rushes player at higher speed on detection	Enter detection radius; confirm charger reaches player faster than patrol enemy
Damage application	Enemy deals the correct HP amount on contact	Let one hit land; confirm HP bar decrements by exactly the GDD value

ENEMY TYPE	BEHAVIOUR TO VERIFY	TEST STEPS
Enemy outside light radius	Enemy is not rendered when outside player light radius	Walk away from an enemy until it exits the visible area; confirm it disappears

Fog-of-War & Light System Tests

CHECK	EXPECTED	HOW TO VERIFY
Radius shrinks over time	Light radius visibly decreases every second at the configured drain rate	Watch HUD light meter during idle play; confirm steady decrease
Light pickup restores radius	Collecting a pickup increases radius by the correct amount	Let radius drain partially; collect one pickup; confirm HUD meter increases
Full darkness = Game Over	Game Over screen triggers when radius reaches 0	Do not collect any pickups; wait for full drain; confirm transition
Darkness overlay renders correctly	Area outside radius is fully dark — no sprite visible beyond the circle	Move player near an enemy just outside radius; confirm enemy invisible
Radius follows player	Light circle stays centred on player during movement	Move quickly in all directions; confirm overlay tracks correctly

Level Transition Tests

CASE	EXPECTED
Exit portal reached — Level 1 to Level 2	VictoryScreen shows time; Level 2 loads with fresh enemy layout and increased drain rate
Player state carries over correctly	HP and light radius values from Level 1 are preserved into Level 2 (or reset — match GDD decision)
Level 2 to final win	Completing the last level shows the final Victory screen — not another level load
Restart from Game Over	Level 1 reloads from start; player stats reset; no leftover entities from previous run
Pause and resume across levels	Pausing just before a transition and resuming does not leave the game in a broken state

Gameplay Balance Checks

BALANCE AREA	TARGET FEEL	ADJUST
Light drain rate	Mild pressure — not instant panic, not ignorable	Players
Enemy patrol speed	Avoidable with attention; threatening if ignored	Players
Charger speed	Feels urgent but fair — player can react	Players
HP count (3 hearts)	Punishing but not one-shot frustrating	Players damage
Light pickup frequency	Rewarding; creates explore-or-die decision	Players more

Bug Log Format — ClickUp / Trello

FIELD	DarkRadius EXAMPLE
Title	Player clips through right wall of corridor on Level 2
Steps	1. Start Level 2 2. Walk east toward narrow right corridor 3. Hold right + up diagonally
Expected	Player stops at wall surface
Actual	Player passes through wall and falls into void
Severity	High / Medium / Low
Assigned To	Name of the teammate fixing it

Phase 5

Deployment

Submission Package

ITEM	WHAT TO INCLUDE
Runnable Build	DarkRadius-1.0.0.jar (or .zip with JAR + assets if needed)
Source Code	GitHub repo link — must be public or shared with instructor
GDD	PDF of our Game Design Document
Diagrams	Architecture, Game Flow, Class Diagram, Level Sketch — PDF or PNG
README	Controls, how to run, known issues, team names, GitHub release link

SDLC Phases at a Glance

PHASE	MAIN OUTPUT	TOOLS
1. Planning	Concept + Roles + Milestones	ClickUp, GitHub (init repo)
2. Design	GDD + 3 Diagrams + Architecture overview	draw.io, Google Docs
3. Build	Working game — iterative, always runnable	libGDX, IntelliJ, GitHub, ClickUp
4. Testing	Bug log, fixed issues, balance notes, clean build	ClickUp tickets, classmate feedback
5. Deployment	Runnable JAR + GitHub Release + README	Gradle, GitHub Releases